人工智能（AI）與機器學習完整學習講義

依據 W3Schools「AI Tutorial」全套教程編寫（繁體中文版）

目錄

編寫說明與材料狀況

1. 人工智能概論
    - 1.1 甚麼是人工智能？
    - 1.2 狹義 AI（Narrow AI / Weak AI）與強 AI（Strong AI）
    - 1.3 機器學習與傳統編程的分別
2. 人工智能的歷史脈絡
    - 2.1 人類智能的起源
    - 2.2 語言的歷史
    - 2.3 數字的歷史
    - 2.4 電腦的歷史
    - 2.5 機械人的歷史
    - 2.6 AI 大事年表
    - 2.7 為甚麼 AI 在當今才興起？
3. 機器學習基礎與術語
    - 3.1 機器學習的定義
    - 3.2 智能決策公式
    - 3.3 機器學習術語（ML Terminology）
    - 3.4 四種學習方式
    - 3.5 學習就是循環（Learning is Looping）
4. 機器學習的數據
    - 4.1 數據佔 ML 項目的八成
    - 4.2 甚麼是數據？
    - 4.3 定量與定性、普查與抽樣
    - 4.4 散點圖與線性圖
5. 神經網絡與感知機
    - 5.1 神經元（Neurons）
    - 5.2 神經網絡（Neural Networks）
    - 5.3 深度學習革命（約 2010 年起）
    - 5.4 感知機（Perceptrons）
    - 5.5 感知機示例：要不要去演唱會？
    - 5.6 感知機的組成部件
6. 模式識別：編程與訓練一個感知機
    - 6.1 模式識別（Pattern Recognition）

- 6.2 如何編程一個感知機
- 6.3 訓練感知機
- 6.4 測試與調校感知機

7. 回歸與聚類
- 7.1 線性回歸（Linear Regressions）
- 7.2 線性函數與線性方程
- 7.3 數據聚類（Data Clusters）
- 7.4 相關係數（Correlation Coefficient）

8. 機器學習的數學基礎
- 8.1 ML 數學總覽
- 8.2 線性代數四大對象
- 8.3 向量（Vectors）
- 8.4 矩陣（Matrices）
- 8.5 張量（Tensors）

9. 機器學習的統計學與概率
- 9.1 統計學的兩大分支
- 9.2 集中趨勢：平均數、中位數、眾數
- 9.3 離散程度：方差與標準差
- 9.4 分佈（Distribution）
- 9.5 概率（Probability）

10. AI 與機器學習的編程語言

11. JavaScript 中的機器學習
- 11.1 為甚麼 JavaScript 適合 ML
- 11.2 JavaScript ML 函式庫
- 11.3 Brain.js 實例

12. TensorFlow.js
- 12.1 甚麼是 TensorFlow.js?
- 12.2 張量與基本操作
- 12.3 模型、層與典型工作流
- 12.4 完整示例的數據處理
- 12.5 TensorFlow.js Visor（tfjs-vis）

13. JavaScript 圖形函式庫

14. AI 應用實例
- 14.1 圖像分類示例
- 14.2 人工智能音樂（Artificial Music Intelligence）
- 14.3 Project Baseline
- 14.4 數據科學家
- 14.5 人工智能醫療（Artificial Health Intelligence）

15. AI 與社會
- 15.1 工作取代（Job Replacement）

15.2 心智理論（Theory of Mind）
16. 重點學習總結
來源列表

編寫說明與材料狀況

- 本講義的內容來源為使用者指定的 W3Schools AI 教程網站（https://www.w3schools.com/ai/，整個網域），涵蓋其中「Machine Learning（ML）」主教程、TensorFlow.js 教程、JavaScript Graphics 教程、History 歷史章節、Job Replacements（工作取代）、Theory of Mind（心智理論）、Mathematics（數學）與 Statistics（統計）等全部章節，共 54 個頁面。
- **關於「附件材料」的說明**：本次請求中提到「attached materials（附件材料）」，但實際並未收到任何附件檔案。因此本講義完全以所列來源（W3Schools AI 教程）為唯一主要參考資料編寫。若附件另行提供，可再據以補充或修訂。
- 凡屬來源支持的內容，均以腳註編號標示對應的 W3Schools 頁面；少數為幫助理解而加入的「補充說明」，會以【補充說明】明確標記，與來源內容區分。
- 原教程內含大量互動示範（Try it Yourself）與網頁程式碼範例，講義中以文字描述其原理與步驟，必要處保留關鍵程式碼。
- 本講義的全部事實性內容均取自 W3Schools AI Tutorial（https://www.w3schools.com/ai/）；文末《來源列表》按正文引用編號列出全部對應頁面。

1. 人工智能概論

1.1 甚麼是人工智能?

人工智能（Artificial Intelligence，AI）指機器能夠模仿人類在以下方面的能力[1]：

- 說話（Talking）
- 思考（Thinking）
- 學習（Learning）
- 規劃（Planning）
- 理解（Understanding）

人工智能又稱為「機器智能」（Machine Intelligence）或「電腦智能」（Computer Intelligence）[1]。

各權威機構對 AI 的定義（均為 2022 年版本，除特別註明）[1]：

來源	定義要旨
Wikipedia	人工智能是機器所展現的智能，有別於人類與動物具備意識與情感的自然智能

來源	定義要旨
Investopedia	指在被編程為像人類般思考、模仿人類行為的機器上進行人類智能的模擬
IBM	利用電腦與機器模仿人類心智的問題解決與決策能力
Britannica	數碼電腦或電腦控制的機器人執行通常與智能生物相關任務的能力，例如推理、發現意義、歸納、從過去經驗中學習

另外，Arthur Samuel 於 1959 年在 IBM Journal of Research and Development（第 3 卷）提出：「機器學習是電腦科學的一個分支領域，讓電腦無需被編程即具備學習能力」（Machine Learning is a subfield of computer science that gives computers the ability to learn without being programmed）[1]。

1.2 狹義 AI（Narrow AI / Weak AI）與強 AI（Strong AI）

人工智能是一個涵蓋多個數據科學領域的科學學科，範圍由狹義 AI 到強 AI，包括機器學習（Machine Learning）、深度學習（Deep Learning）、大數據（Big Data）與數據挖掘（Data Mining）[1]。

狹義 AI（Narrow Artificial Intelligence） 局限於狹窄（特定）範圍，現今我們周圍大部分 AI 都屬此類，例如[1]：

- 電郵垃圾郵件過濾器、文字轉語音（Text to Speech）、語音識別（Speech Recognition）
- 自動駕駛汽車、電子支付、Google Maps、文字自動更正、自動翻譯
- 聊天機器人（Chatbots）、社交媒體人臉偵測、視覺感知、搜索算法、機械人
- 自動投資、自然語言處理（NLP）、飛行無人機（Drones）
- IBM 的 Dr. Watson、Apple 的 Siri、Microsoft 的 Cortana、Amazon 的 Alexa、Netflix 的推薦系統

狹義 AI 又稱**弱 AI（Weak AI）**。兩者的分別[1]：

- 弱 AI（Weak AI）：為「模擬」（simulate）人類智能而建。
- 強 AI（Strong AI）：為「複製」（copy）人類智能而建。

強 AI（Strong Artificial Intelligence） 是模仿人類智能的 AI 類型，代表具備思考、規劃、學習與溝通的能力，是理論上的下一個層次，即「真正智能」（True Intelligence），朝著具備自我意識（self-awareness）、意識（consciousness）與客觀思想（objective thoughts）的機器方向發展[1]。

教程並引用 Alan Turing（圖靈）的觀點：「我們無需判定機器能否『思考』，只需判定機器能否像人類一樣聰明地行動。」（One need not decide if a machine can "think". One need only decide if a machine can act as intelligently as a human.）[1]

1.3 機器學習與傳統編程的分別

傳統編程使用算法（algorithms）從數據產生結果；機器學習則從數據與結果創造算法[2]：

- **傳統編程**：數據＋算法＝結果（Data + Algorithms = Results）
- **機器學習**：數據＋結果＝算法（Data + Results = Algorithms）

深度學習（Deep Learning）是機器學習的子集，其對照如下[2]：

機器學習（Machine Learning）	深度學習（Deep Learning）
AI 的子集	機器學習的子集
使用較小的數據集	使用較大的數據集
由人類訓練	自行學習
創造簡單算法	創造複雜算法

【補充說明】簡單而言：AI 是最大的概念圈子，機器學習是其中的子集，深度學習又是機器學習的子集；三層關係為 AI ⊃ ML ⊃ DL。

2. 人工智能的歷史脈絡

2.1 人類智能的起源

約 70,000 年前，人類大腦發生了某種變化，開始發展「認知智能」（Cognitive Intelligence）：能夠理解語言、理解數字、進行抽象思考[3]。

- **文字與數字**：使用文字是智能發展的一大步——「大象」（Elephant）比「大動物」（Big Animal）傳遞更多信息；「5」或「50」比「幾個」或「很多」更精確[3]。
- **語言**：「大橡樹後面有一頭獅子」比大喊「危險！」傳遞更多信息。擁有語言很可能是人類區別於動物的關鍵特徵[3]。
- **抽象思考**：對非具體事物（如自由、意念、概念）的思考。日常許多行為都運用抽象思考：使用概念、理論、隱喻、類比、想像[3]。

關於「智能是甚麼」，教程指出並無公認的定義，但可定義為：能夠理解抽象思考、理解語言、理解數字的能力；亦可定義為知識的集合。Yampolsky（2013）：「智能可定義為以高效方法獲取知識的過程。」（Intelligence could be defined as the process

of obtaining knowledge by efficient means.）AI 的挑戰正在於理解智能是甚麼、如何運作[3]。

教程並提出「人類大腦是否優越？」的問題：鳥類腦很小，卻知道秋天不宜育雛何時南飛；蜜蜂腦更小，卻懂得以舞蹈溝通花蜜位置；植物沒有腦，卻知道如何繁殖與準備過冬[3]。

2.2 語言的歷史

語言是結構化的溝通系統。猿類、鯨魚、鳥類、蜜蜂都會溝通，但只有人類發展出真正的語言——以一套詞彙（含動詞與名詞）構成句子來表達意念，而且連兒童都能掌握這技能[4]。

Hauser、Chomsky 與 Fitch（2002）提出語言發展的六個方面[4]：

1. 心智理論（Theory of Mind）的理解
2. 聲音信號（Vocal Signals）的理解
3. 模仿（Imitation）的理解
4. 數字（Numbers）的理解
5. 意向性溝通（Intentional Communication）的理解
6. 非語言表徵（Non-linguistic Representations）的理解

人類語言由有限的「詞彙」（Words）組合成「句子」（Sentences）；電腦語言同樣以有限的詞彙組合成「陳述句」（Statements）[4]。

已知最早的書面語言為埃及文與蘇美爾文（今日仍在使用最古老的書面語言是中文）。教程列出的年代（公元前以 BC、公元以 AC 表示）[4]：

年代	語言
3500 BC	蘇美爾文（Sumerian）
3000 BC	埃及文（Egyptian）
1500 BC	中文（Chinese）
1500 BC	吠陀梵文（Vedic Sanskrit）
1500 BC	希臘文（Greek）
1000 BC	希伯來文（Hebrew）
900 BC	阿拉姆文（Aramaic）
700 BC	伊特魯裡亞文（Etruscan）
500 BC	泰米爾文（Tamil）
700 BC	拉丁文（Latin）

年代	語言
700 AC	古典梵文（Classical Sanskrit）
500 AC	阿拉伯文（Arabic）
400 AC	哥德文（Gothic，德語）
700 AC	英文（English）
700 AC	日文（Japanese）
800 AC	法文（French）
900 AC	意大利文（Italian）
1000 AC	西班牙文（Spanish）

（注意：上表照錄原教程內容；原文中拉丁文標為 700 BC、伊特魯裡亞文 900 BC 的排序與常見史料略有出入，此處忠實保留來源寫法，未作改動。）

2.3 數字的歷史

人工智能本質上關於數字（Numbers）與計數（Counting）。AI 是人類計算需求的產物；動物研究顯示動物只能理解非常簡單的計數[5]。

巴比倫六十進制（Sexagesimal，Base 60）：巴比倫人啟動了複雜計數的發展，其數字系統有 60 個不同數碼。60 的優點是能被 1、2、3、4、5、6、10、12、15、20、30、60 整除。它是位值（place-value）系統，左邊的數碼代表完整值，例如「1,5」表示 65（1×60＋5），「3,30」表示 210（3×60＋30）[5]。

巴比倫系統遺留至今的影響[5]：

甚麼	數值	原因
一個完整圓周	360°	6 × 60 = 360
半個圓周	180°	3 × 60 = 180
一小時	60 分鐘	1 × 60

巴比倫人觀察到一年約 360 天（早期太陽曆），將天空分為 12 個黃道星座、每個跨 30 度；60 分鐘一小時、60 秒一分鐘的時間度量亦源於此系統[5]。

十二進制（Dozenal，Base 12）：一打（dozen）有 12 個、一日有 12 小時、一夜有 12 小時、一年有 12 個月。12 可被 1、2、3、4、6、12 整除。用雙手可數到 60：每隻手指有 3 個關節，拇指在左手數到 12，右手記錄左手完成的次數（1 隻滿手＝12、2 隻＝24、3 隻＝36、4 隻＝48、5 隻＝60）[5]。

羅馬數字（Base 10）：源於羅馬、在歐洲使用至中世紀。符號與數值：I＝1、V＝5、X＝10、L＝50、C＝100、D＝500、M＝1000。羅馬人使用十進制，可能因為以 10 隻手指計數。今日世界大部分地區都用十進制（西方、中國、婆羅米、希臘、希伯來數字等）[5]。

數碼數字（Digital Numbers，Base 2）：人類可以用十進制計數，電腦卻不能——電力無法計數，只有「開」（On）與「關」（Off）兩態。電腦以電力或磁力代表開與關，使用二進制數（Binary Numbers），例如 11100110（相對於十進制的 230）[5]。

2.4 電腦的歷史

電腦演進的目標始終是「減少計算所需時間」[6]：

- **算盤（Abacus）**：巴比倫算盤約於公元前 2700–2300 年首次出現，以連續列定義 60 進位數字系統。羅馬算盤使用 10 個羅馬數碼，是世界上第一部手提電腦，為羅馬工程師、商人與稅吏所用[6]。
- **模擬電腦（Analog Computers）**：Charles Babbage 的差分機（Difference Engine，1822）為減少複雜數學函數計算時間而設計的機械機器；分析機（Analytical Engine，1833）已具備現代電腦元素——算術、邏輯與記憶體。兩者都以十進制機械齒輪運算[6]。
- **數碼電腦（Digital Computers）**：以 0/1 開關運算。第一部電動數碼電腦由德國的 Konrad Zuse 於 1941 年設計建造，使用 2,600 個電繼電器（electrical relays）作 0/1 開關，時鐘速度約 5 Hz[6]。
- **電子電腦（Electronic Computers）**：第一代（1945–1950）用真空管（vacuum tubes）作二進制開關，時鐘速度介乎 500 KHz 與 1 MHz；第二代（1950–1960）用晶體管（transistors），比真空管快得多；第三代（1960）用集成電路（integrated circuits），比晶體管快得多[6]。

電腦速度演進（每秒指令數）[6]：

年份	電腦	每秒指令數	每指令位元數
1941	Z3	5	4
1945	ENIAC	5,000	8
1981	IBM PC	5,000,000	16
1995	Intel Pentium PC	100,000,000	32
2000	AMD PC	1,000,000,000	64
2020	iPhone 12	10,000,000,000	128

教程指出：若相信摩爾定律（Moore's Law），按 Ray Kurzweil 的估計，2020 年電腦具備老鼠大腦的容量，2030 年電腦可能具備人類大腦的容量[6]。並引用畢加索（Pablo Picasso）名言：「電腦毫無用處，因為它們只能給你答案。」（Computers are useless because they can only give you answers.）[6]

2.5 機械人的歷史

- 工業機械人已存在超過 50 年。第一項機械人專利於 1954 年申請、1961 年批出[7]。
- 1969 年 Victor Scheinman 發明史丹福手臂（Stanford Arm，史丹福大學）；1972 年為麻省理工學院人工智能實驗室設計 MIT Arm。這些「機械人」不被視為智能——它們是可作 6 軸手臂運動的電機，但新設計讓機器可跟隨編程路徑，開啟汽車噴漆、焊接與裝配等「機械人工作」[7]。
- 先驅機械人公司 Unimation（獲通用汽車支持）將 Scheinman 的設計以 PUMA（Programmable Universal Machine for Assembly，可編程通用裝配機）推出市場[7]。
- 大部分工業機械人是**非智能**的。現代機械人多是自主或半自主（autonomous / semi-autonomous），因為編程後無需太多人類輸入。機械人可被編程做很多不同的事，但永遠不會自行改變正在做的事——它會持續同一工作直至被關掉[7]。
- **重要觀念**：機械人與人工智能是兩回事，機械人技術不是 AI 的子集。機械人是實體事物；50 年發展後，幾乎任何東西都可編程——收音機、手錶、電話、以至機械人[7]。
- **AI 機械人**：AI 可以內建於機械人。Hanson Robotics 的 Sophia 體現了對 AI 未來的部分夢想——她是科學、工程與藝術的結合，一個由人類打造的科幻角色，描繪 AI 與機械人學的未來。MIT 的 Kismet 被編程理解身體語言與語音抑揚，其創作者研究人類與嬰兒如何以語氣與視覺線索互動，Kismet 可成為擬人學習系統的基礎[7]。

2.6 AI 大事年表

教程列出的 AI 與 ML 歷史年表[8]：

年份	事件
1950	Alan Turing 發表《Computing Machinery and Intelligence》
1952	Arthur Samuel 開發自學習的跳棋（checkers）程式
1956	John McCarthy 在會議上使用「Artificial Intelligence」一詞

年份	事件
1957	第一部用於數值與科學計算的編程語言（FORTRAN）
1958	第一部 AI 編程語言（LISP）
1959	Arthur Samuel 使用「Machine Learning」一詞
1959	John McCarthy 與 Marvin Minsky 創立 MIT 人工智能計劃
1961	第一部工業機械人（Unimate）在通用汽車裝配線上運作
1965	Joseph Weizenbaum 的 ELIZA 成為首個可就任何話題溝通的程式
1972	第一部邏輯編程語言（PROLOG）
1991	美軍在海灣戰爭使用 DART（自動後勤規劃與調度）
1997	IBM 深藍（Deep Blue）擊敗國際象棋世界冠軍
2002	首部掃地機械人（Roomba）
2005	自動駕駛汽車（STANLEY）贏得 DARPA 大賽
2008	語音識別突破（Google）
2011	神經網絡在交通標誌識別上勝過人類（99.46% 對 99.22%）
2011	Apple Siri
2011	IBM Watson 贏得問答節目 Jeopardy!
2014	Amazon Alexa
2014	Microsoft Cortana
2014	自動駕駛汽車（Google）通過州駕駛考試
2015	Google AlphaGo 在圍棋擊敗多位人類冠軍
2016	Hanson Robotics 的擬人機械人 Sofia

2.7 為甚麼 AI 在當今才興起?

John McCarthy 被廣泛認可為「人工智能之父」，於 1950 年代中期創造「Artificial Intelligence」一詞，並定義為「製造智能機器的科學」（the science of making intelligent machines）[8]。

算法自那時已存在，AI 到近年才更受注目，原因是過往[8]：

- 計算能力不夠強
- 電腦儲存不夠大
- 大數據不可得
- 快速互聯網不可得

另一股強大推動力是大型企業（Google、Microsoft、Facebook、YouTube）的重大投資——因為它們的數據集已大到無法以傳統方式處理[8]。

教程以「人 vs 機器」對照總結：人——聰明、慢、不精確；電腦——愚笨、快、精確[8]。並列出研究 AI 引發的問題：「電腦能否像人類般思考？」「電腦會否比人類更聰明？」「電腦會否接管世界？」教程指出：機器已能理解口頭指令、識別面孔、駕駛汽車、比我們更擅長玩遊戲[8]。

教程另附一節 AI 電影列表（由 1927 年《Metropolis》至 2020 年《Coded Bias》），作為 AI 文化影響的延伸閱讀，包括《2001: A Space Odyssey》（1968）、《Star Wars》（1977）、《Blade Runner》（1982）、《The Matrix》（1999）、《Her》（2013）、《The Imitation Game》（2014，片名出自 Turing 1950 年論文提出的「模仿遊戲」）、《Ex Machina》（2015）、HBO 劇集《Westworld》（2016）、紀錄片《iHuman》（2019）與《Coded Bias》（2020，探討 AI 內嵌偏見）等[8]。

3. 機器學習基礎與術語

3.1 機器學習的定義

機器學習（Machine Learning，ML）常被誤認為等同於人工智能——這並不正確，機器學習是 AI 的子集，是一門利用數據教導機器的學科[9]。Arthur Samuel（1959）的經典定義：「機器學習是一個研究領域，讓電腦無需被編程即具備學習能力。」[9]

Tom Michael Mitchell（1951 年生）是美國電腦科學家、卡內基梅隆大學（CMU）教授、曾任 CMU 機器學習系系主任，其 1999 年的學習定義[9]：

> 「若一個電腦程式在某類任務 T 上、以表現度量 P 衡量的表現，能隨經驗 E 而改進，則稱該程式能從經驗 E 學習。」（A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.）

其中 E＝經驗（次數）、T＝任務（如駕駛汽車）、P＝表現（好或壞）[9]。

傳統計算與機器學習的對照[9]：

- **傳統計算**：數據＋電腦算法＝結果（Data + Computer Algorithm = Result）
- **機器學習**：數據＋結果＝電腦算法（Data + Result = Computer Algorithm）

3.2 智能決策公式

教程列出電腦作出智能決策的循環步驟[9]：

7. 保存所有行動的結果
8. 模擬所有可能的結果
9. 將新行動與舊行動比較
10. 檢查新行動是好是壞
11. 若新行動「較不壞」則選擇它
12. 不斷重複以上步驟

電腦能夠把這循環執行數百萬次，這已證明電腦能作出非常智能的決策[9]。

3.3 機器學習術語（ML Terminology）

關係（Relationships）：機器學習系統利用輸入之間的關係來產生預測（Predictions）。代數中關係常寫作 y = ax + b；機器學習中則寫作 **y = b + wx**[10]：

代數寫法	機器學習寫法	意義
y	y	標籤（label）——我們想預測的東西
a	w	權重（weight）／斜率（slope）
x	x	特徵（features）——輸入值
b	b	截距（intercept）

有時有多個特徵、各有不同權重：$\mathbf{y = b + w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4}$[10]。

標籤（Labels）：我們想預測的東西，猶如線性圖中的 y[10]。

特徵（Features）：輸入值，猶如線性圖中的 x[10]。

模型（Models）：模型定義標籤（y）與特徵（x）之間的關係。模型生命週期有三個階段：數據收集（Data Collection）、訓練（Training）、推斷（Inference）[10]。

訓練（Training）：訓練的目標是創造一個能回答問題的模型，例如「一間房屋的預期價格是多少？」[10]

推斷（Inference）：以訓練好的模型，用即時數據推斷（預測）數值，即把模型投入生產[10]。

機器學習兩大階段[10]：

13. 訓練（Training）：以輸入數據計算模型的參數。
14. 推斷（Inference）：「已訓練」的模型對任何輸入輸出正確數據。

3.4 四種學習方式

學習方式	要點
監督學習（Supervised Learning）	使用一組輸入變量預測輸出變量的值；以帶標籤數據（已知答案的數據）訓練算法進行「分類」與「預測結果」。例子：根據已知垃圾郵件例子分類「甚麼是垃圾郵件」；根據你播放過的影片預測你喜歡哪類影片[10]
無監督學習（Unsupervised Learning）	從任何無標籤數據集找出模式（或分組），預測未定義的關係；目標是創造能自我改進的電腦算法。教程預期機器學習將轉向無監督學習，讓程式員無需建立模型也能解決問題[10]
強化學習（Reinforcement Learning）	基於無監督學習，但會接收用戶對決策好壞的反饋，反饋有助改進模型[10]
自監督學習（Self-Supervised Learning）	與無監督學習相似，同樣使用沒有人工標籤的數據；分別在於無監督學習使用聚類、分組與降維，而自監督學習自行得出結論，用於回歸與分類任務[10]

3.5 學習就是循環（Learning is Looping）

ML 模型通過對數據多次循環（Looping）來訓練；每次迭代都會調整權重值（Weight Values）；當迭代無法再降低成本（Cost）時，訓練即告完成[11]。

梯度下降（Gradient Descent） 是解決 AI 問題的常用算法。教程以簡單線性回歸模型示範：目標是為一組 (x,y) 點擬合一條線性圖。這可以用數學公式求解，但機器學習算法同樣能解決——從散點圖與線性模型 **y = wx + b** 開始，透過改變線的權重（斜率 w）與偏置（截距 b）訓練模型找出最擬合的線[11]。

成本函數（Cost Function）（又稱誤差函數，Error Function）：解決回歸問題的標準方法，用以衡量解的優劣。函數使用模型的權重與偏置（y = wx + b），根據線與數據的擬合程度返回誤差。計算方法：循環遍歷圖上所有 (x,y) 點，將每個點的 y 值與線之間的距離平方後求和；取平方是為確保正值、並讓誤差函數可微分（differentiable）[11]。

成本函數公式中各符號的意義[11]：

- E：誤差（成本）
- N：觀測（點）的總數
- y：每個觀測的值（標籤）

- x：每個觀測的值（特徵）
- m：斜率（權重）
- b：截距（偏置）
- mx + b：預測值
- 1/N × Σ：平方平均值（mean squared value）

訓練函數（Train Function）：執行梯度下降，讓算法沿成本函數走向最佳的線；每次迭代都把 m 與 b 更新到成本（誤差）更低的線。訓練函數對所有數據循環多次，並以兩個偏導數（partial derivatives）計算移動方向，更新權重與偏置[11]。

教程示範的 Trainer 物件：接受兩個數組（xArr, yArr）中任意數量的 (x,y) 值；權重（weight）初始設為 0、偏置（bias）設為 1；並須設定學習常數（learnc）與定義成本變量[11]。

4. 機器學習的數據

4.1 數據佔 ML 項目的八成

一個機器學習項目中，高達 80% 的工作與「數據收集」有關，需要思考：需要甚麼數據？有哪些數據可用？如何選擇、收集、清洗、準備與使用數據[12]？

4.2 甚麼是數據?

在機器學習中，數據是事實（facts）的集合[12]：

類型	例子
數字（Numbers）	價格、日期
度量（Measurements）	尺寸、高度、重量
詞語（Words）	名字與地點
觀察（Observations）	數算汽車
描述（Descriptions）	「天氣很冷」

人類智能需要數據（地產經紀需要已售房屋數據來估價），AI 同樣需要數據（ML 程式需要數據來估價）。數據幫助我們看見與理解、發掘新機會、化解誤解[12]。

各領域應用[12]：

- **醫療**：醫療與生命科學收集公共衛生數據與病人數據，學習如何改善病人護理、拯救生命。

- **商業**：許多行業最成功的公司都以數據驅動，使用精密的數據分析學習如何表現更佳。
- **金融**：銀行與保險公司收集並評估客戶、貸款與存款數據，以支持策略決策。

4.3 定量與定性、普查與抽樣

定量數據（Quantitative） 是數值性的：55 輛車、15 米、35 個小孩。**定性數據（Qualitative）** 是描述性的：「很冷」、「很長」、「很有趣」[12]。

普查（Census）：收集群體中每個成員的數據——準確，但難以執行。**抽樣（Sample）**：收集部分成員的數據——不準確，但較易執行。例子：想知道多少美國人吸煙，可以問每一個美國人（普查），或問 10,000 人（抽樣）[12]。

抽樣術語[12]：

- 總體（Population）：我們想收集信息的個體（對象）群組。
- 普查（Census）：總體中每個個體的信息。
- 樣本（Sample）：總體一部分的信息（以期代表全體）。

隨機樣本（Random Sample）：樣本要能代表總體，必須隨機收集——總體中每個成員都有相等機會出現在樣本中。**抽樣偏差（Sampling Bias / Error）**：當樣本收集方式令某些個體較少（或較多）機會被納入時發生[12]。

大數據（Big Data）：沒有先進機器協助、人類無法處理的數據。大數據沒有以大小作出的定義，但隨著我們不斷收集更多數據、儲存成本不斷下降，數據集正變得越來越大[12]。

數據挖掘（Data Mining）：大數據帶來複雜的數據結構，大數據處理的很大一部分是提煉（refining）數據[12]。

4.4 散點圖與線性圖

教程以「房屋價格對面積」的數據作貫穿示例[13]：

價格（Price）	**7**	**8**	**8**	**9**	**9**	**9**	**10**	**11**	**14**	**14**	**15**
面積（Size）	50	60	70	80	90	100	110	120	130	140	150

散點圖（Scatter Plot）：以散佈在區域上的點表示兩個數值之間的關係。適合用於[13]：看清「大局」（Big Picture）、比較不同數值、發現潛在趨勢、發現數據中的模式、發現數據之間的關係、發現聚類與相關性。

線性圖（Linear Graphs）：機器學習常用線性圖顯示關係，線性圖展示線性函數 **y = ax + b** 的值。關鍵詞[14]：

- 線性（Linear）＝直（Straight）：線性圖是一條直線，由 x 軸（水平）與 y 軸（垂直）構成。
- 斜率（Slope）＝角度（Angle）：即 y = ax 中的 a 值（教程例子中 slope = 1.2）。
- 截距（Intercept）＝起始值（Start value）：即 y = ax + b 中的 b 值（教程例子中 slope = 1.2、intercept = 7）。

5. 神經網絡與感知機

5.1 神經元（Neurons）

科學家同意人腦有 800 億至 1,000 億個神經元（neurons），神經元之間有數以千億計的連接。神經元（又稱神經細胞，Nerve Cells）是大腦與神經系統的基本單位，負責接收來自外部世界的輸入、發出輸出（對肌肉的指令）、並在中間轉換電信號[9]。

5.2 神經網絡（Neural Networks）

人造神經網絡（Artificial Neural Networks）通常簡稱神經網絡（NN）[9]。教程對神經網絡的描述[2]：

- 是一種編程技術（programming technique）
- 是機器學習中使用的方法
- 是從錯誤中學習的軟件

神經網絡基於人類大腦的運作方式：神經元互相發送消息；當神經元反覆嘗試解決問題時，會強化導致成功的連接、削弱導致失敗的連接[2]。

神經網絡實際上是**多層感知機（Multi-Layer Perceptrons）**，是深度學習的本質（essence）、是歷史上最重要的發現之一。神經網絡能解決「無法以算法解決」的問題[9]：

- 醫療診斷（Medical Diagnosis）
- 人臉偵測（Face Detection）
- 語音識別（Voice Recognition）

神經網絡模型：最簡單的神經網絡由輸入層（input layer，黃色）、隱藏層（hidden layer，藍色）與輸出層（output layer，紅色）組成。輸入數據先對照隱藏層處理，再產生最終輸出[2]：

- 第一層：黃色感知機基於輸入作簡單決策，每個決策送給下一層的感知機。

- 第二層：藍色感知機通過衡量第一層的結果作決策，在比第一層更抽象的層面作出更複雜的決策。

深度神經網絡（Deep Neural Networks）：由多個隱藏層組成、對海量數據執行複雜運算；每一後繼層以前一層為輸入。例如光學閱讀（optical reading）以低層識別邊緣、以高層識別字母。在深度神經網絡模型中，輸入數據（黃）經隱藏層（藍）處理、再經更多隱藏層（綠）修改，產生最終輸出（紅）；第三層的綠色感知機作出更複雜的決策[2]。

5.3 深度學習革命（約 2010 年起）

深度學習革命約於 2010 年開始，此後深度學習解決了許多「無法解決」的問題。這場革命並非由單一的發現啟動，而是多項必要因素同時就緒[9]：

- 電腦夠快
- 電腦儲存夠大
- 發明了更好的訓練方法
- 發明了更好的調校（tuning）方法

長頸鹿故事（The Giraffe Story）：2015 年，倫敦帝國學院學生 Matthew Lai 創造了名為 Giraffe 的神經網絡，經 72 小時訓練即可以國際大師（international master）水準下國際象棋。電腦下棋並非新聞，但這個程式的創造方式是新的——聰明的下棋程式要花數年建造，而 Giraffe 只用 72 小時以神經網絡建成[9]。

5.4 感知機（Perceptrons）

感知機是**人造神經元（Artificial Neuron）**、是最簡單的神經網絡，是機器學習的組成積木[15]。

Frank Rosenblatt（1928–1971）：美國心理學家，AI 領域的重要人物。1957 年，他在康奈爾航空實驗室（Cornell Aeronautical Laboratory）的 IBM 704 電腦上「發明」了感知機程式。當時科學家已發現腦細胞（神經元）以電信號接收感官輸入，並以電信號儲存信息、基於先前輸入作出決策。Frank 的想法是：感知機可以模擬大腦原理，具備學習與決策能力[15]。

原始感知機的設計：接收多個二進制輸入，產生一個二進制輸出（0 或 1）；以不同的**權重（weights）**代表每個輸入的重要性，數值總和須大於**閾值（threshold）**才作出「是／否」（true/false，1/0）的決策[15]。

5.5 感知機示例：要不要去演唱會？

想像你腦中有一個感知機，要決定你是否應該去一場演唱會。各準則的輸入與權重[15]：

準則	輸入	權重
藝人好（Artists is Good）	x1 = 0 或 1	w1 = 0.7
天氣好（Weather is Good）	x2 = 0 或 1	w2 = 0.6
朋友會來（Friend will Come）	x3 = 0 或 1	w3 = 0.5
有食物供應（Food is Served）	x4 = 0 或 1	w4 = 0.3
有酒供應（Alcohol is Served）	x5 = 0 或 1	w5 = 0.4

感知機算法（Frank Rosenblatt 提出）四步[15]：

15. 設定閾值（Threshold = 1.5）
16. 將所有輸入乘以各自的權重：x1×w1 = 1×0.7 = 0.7；x2×w2 = 0×0.6 = 0；x3×w3 = 1×0.5 = 0.5；x4×w4 = 0×0.3 = 0；x5×w5 = 1×0.4 = 0.4
17. 將所有結果相加：0.7 + 0 + 0.5 + 0 + 0.4 = 1.6（加權和，Weighted Sum）
18. 激活輸出：若總和 > 1.5 則返回 true（「好，我會去演唱會」）

注意：若天氣對你的權重是 0.6，對別人可能不同——較高的權重代表天氣對他們更重要；若你的閾值是 1.5，別人可能不同——較低的閾值代表他們更願意去任何演唱會[15]。

5.6 感知機的組成部件

拆解感知機的理論組成[15]：

19. **輸入（Inputs / Nodes）**：感知機接收一個或多個輸入，輸入稱為「節點」（nodes），節點同時有「值」與「權重」。
20. **節點值（Input Values）**：輸入節點的值為二進制 1 或 0，可解讀為 true/false、yes/no（例子中的值：1, 0, 1, 0, 1）。
21. **節點權重（Node Weights）**：賦予每個輸入的值，顯示每個節點的強度；值越高，輸入對輸出的影響越強（例子中的權重：0.7, 0.6, 0.5, 0.3, 0.4）。
22. **求和（Summation）**：感知機計算輸入的加權和——每個輸入乘以對應權重再求和：0.7×1 + 0.6×0 + 0.5×1 + 0.3×0 + 0.4×1 = 1.6。
23. **閾值（Threshold）**：感知機「發火」（fire，輸出 1）所需的值，否則保持非激活（輸出 0）。例子中閾值為 1.5。
24. **激活函數（Activation Function）**：求和後應用，目的是為輸出引入非線性（non-linearity），根據匯總輸入決定感知機是否發火。例子中的激活函數很簡單：(sum > threshold)，即 (1.6 > 1.5)。

25. **輸出（The Output）**：激活函數的結果，代表感知機基於輸入與權重的決策或預測；激活函數把加權和映射為二進制值（1 或 0，可解讀為 true/false、yes/no）。本例輸出為 1，因為 (sum > threshold) == true。

感知機學習（Perceptron Learning）：感知機可通過「訓練」（training）從例子中學習。訓練期間，感知機基於觀察到的誤差調整權重，通常使用感知機學習規則（perceptron learning rule）或反向傳播（backpropagation）等學習算法。學習過程向感知機呈現帶標籤的例子（已知期望輸出），感知機比較自己的輸出與期望輸出並相應調整權重，目標是最小化預測輸出與期望輸出之間的誤差，從而學會能對新的未知輸入作出準確預測的權重[15]。

注意：決策顯然不能只靠一個神經元作出，其他神經元必須提供更多輸入。「多層感知機」（Multi-Layer Perceptrons）可用於更精密的決策。感知機雖然對人造神經網絡的發展影響深遠，但只能學習**線性可分（linearly separable）**的模式；然而，把多個感知機分層堆疊並引入非線性激活函數，神經網絡就能克服此限制、學習更複雜的模式[15]。

感知機與神經網絡的關係：感知機定義了進入神經網絡的第一步，常被用作更複雜神經網絡（如多層感知機 MLP、深度神經網絡 DNN）的積木。把多個感知機分層組合並連接成網絡結構，這些模型就能學習並表示數據中複雜的模式與關係，實現圖像識別、自然語言處理與決策等任務[15]。

6. 模式識別：編程與訓練一個感知機

6.1 模式識別（Pattern Recognition）

神經網絡應用於人臉識別（Facial Recognition）等場景，這些應用使用**模式識別**。這類分類（Classification）可以用感知機完成——感知機可把數據分為兩部分，故又稱**線性二分類器（Linear Binary Classifiers）**[16]。

模式分類（Pattern Classification）：想像在散佈著 x、y 點的空間中有一條直線（線性圖）。如何把點分類為「線上」與「線下」？感知機可以在不知道直線公式的情況下，被訓練去識別線以上的點[16]。

6.2 如何編程一個感知機

教程以 JavaScript 示範，步驟如下[16]：

26. 建立一個簡單的繪圖器（plotter）
27. 建立 500 個隨機 x、y 點
28. 顯示這些 x、y 點
29. 建立線函數 f(x)

30. 顯示這條線
31. 計算「期望答案」（desired answers）
32. 顯示期望答案

其中「計算正確答案」的規則：線函數為 **y = x × 1.2 + 50**；若點在線上，期望答案為 1；若點在線下，期望答案為 0；期望答案儲存在數組 desired[] 中。顯示時：desired[i] = 1 顯示黑點，否則顯示藍點[16]。

6.3 訓練感知機

訓練任務：想像散佈 x、y 點空間中的一條直線，訓練感知機把點分類到線上與線下[17]。三個步驟：建立感知機物件、建立訓練函數、以正確答案訓練感知機。

建立感知機物件（Perceptron Object）：接受兩個參數——輸入數量（no）與學習率（learningRate，預設 0.00001），並為每個輸入建立介乎 -1 與 1 之間的隨機權重[17]：

```
// Perceptron Object
function Perceptron(no, learningRate = 0.00001) {
  // Set Initial Values
  this.learnc = learningRate;
  this.bias = 1;
  // Compute Random Weights
  this.weights = [];
  for (let i = 0; i <= no; i++) {
    this.weights[i] = Math.random() * 2 - 1;
  }
}
```

關鍵概念[17]：

- **隨機權重（Random Weights）**：感知機以每個輸入的隨機權重開始。
- **學習率（Learning Rate）**：訓練時每犯一個錯誤，權重會以一個小分數調整，這個小分數就是「感知機的學習率」（在物件中稱為 learnc）。
- **偏置（Bias）**：有時兩個輸入都是零，感知機可能產生錯誤輸出；為避免此事，給感知機一個值恆為 1 的額外輸入，稱為偏置（bias）。

激活函數（Activate Function）：把每個輸入乘以權重、求和、計算結果；總和大於 0 輸出 1，否則輸出 0[17]：

```
this.activate = function(inputs) {
  let sum = 0;
  for (let i = 0; i < inputs.length; i++) {
    sum += inputs[i] * this.weights[i];
  }
  if (sum > 0) {return 1} else {return 0}
```

```
}
```

訓練函數（Training Function）：基於激活函數猜測結果；每猜錯一次，感知機就調整權重；經過多次猜測與調整後，權重會趨向正確[17]：

```
this.train = function(inputs, desired) {
  inputs.push(this.bias);
  let guess = this.activate(inputs);
  let error = desired - guess;
  if (error != 0) {
    for (let i = 0; i < inputs.length; i++) {
      this.weights[i] += this.learnc * error * inputs[i];
    }
  }
}
```

反向傳播（Backpropagation）：每次猜測後，感知機計算猜錯了多少；若猜錯，就調整偏置與權重，令下次猜測「更接近正確一點」。這種學習方式稱為反向傳播。嘗試幾千次後，感知機會變得相當擅長猜測[17]。

教程的完整使用示例：以 500 個點（numPoints = 500）、學習率 0.00001、線函數 f(x) = x × 1.2 + 50，建立 Perceptron(2, learningRate)，然後以雙重循環訓練 10,000 輪（外層 j ≤ 10000，內層遍歷所有點 ptron.train([xPoints[i], yPoints[i]], desired[i])），最後以 ptron.activate 猜測並把 guess == 0 的點顯示為藍色、其餘為黑色[17]。

6.4 測試與調校感知機

感知機必須被**測試與評估**，且必須以**真實值（Real Values）**測試：產生新的未知點，檢查感知機能否猜對答案；並可加入計數器統計錯誤次數[18]。

調校（Tune）感知機的建議[18]：

- 調整學習率
- 增加訓練數據的數量
- 增加訓練迭代的次數

7. 回歸與聚類

7.1 線性回歸（Linear Regressions）

回歸（Regression） 是確定一個變量（y）與其他變量（x）之間關係的方法。在統計學中，線性回歸是對 y 與 x 之間線性關係建模的方法；在機器學習中，線性回歸是一種**監督式**機器學習算法[19]。

從散點數據預測未來價格的三種途徑[19]：

33. 使用手繪線性圖
34. 建立線性關係模型（例子中斜率 a 為計算出的平均值、截距 = 0）
35. 建立線性回歸模型（使用線性回歸函數）

線性圖可寫作 **y = ax + b**，其中 y 是想預測的價格、a 是線的斜率、x 是輸入值、b 是截距[19]。

多項式回歸（Polynomial Regression）：若散點數據不擬合線性回歸（一條穿過各點的直線），數據可能擬合多項式回歸。多項式回歸與線性回歸一樣，利用變量 x 與 y 之間的關係，找出穿過數據點畫線的最佳方式[19]。

7.2 線性函數與線性方程

函數（Function） 是每個輸入都對應一個輸出的特殊關係，常寫作 f(x)。例如 f(x) = x 時，x=1→y=1、x=2→y=2…… ；f(x) = 2x 時，x=1→y=2、x=2→y=4、x=3→y=6、x=4→y=8、x=5→y=10[20]。

線性方程（Linear Equation） 是直線的方程，例如：y = x、y = x×2、y = x×2 + 7、y = ax + b、5x = 3y、y/2 = 6。**非線性方程**不能含有指數或平方根，例如 $y = x^2$、y = Math.sqrt(x)、y = Math.sin(x) 均為非線性[20]。

線性回歸：通過把線性圖擬合到數據來為兩個變量的關係建模；一個變量（x）視為數據，另一個（y）視為依賴變量。例如以線性回歸建立房屋價格與面積的關係模型[20]。

線性最小二乘法（Linear Least Squares，LLS）：線性代數用於求解線性方程；LLS 是一組解決線性回歸相關統計問題的公式[20]。

7.3 數據聚類（Data Clusters）

要點[21]：

- 聚類（Clusters）是相似數據的集合
- 聚類分析（Clustering）是無監督學習的一種
- 相關係數（Correlation Coefficient）描述關係的強度

聚類：基於相似性的數據集合；在圖中聚在一起的數據點常可分類為聚類（教程示例圖中可分辨 3 個不同的聚類）。聚類蘊含大量有價值信息，但形狀各異，識別聚類的兩大方法是[21]：

36. 使用可視化（Visualization）
37. 使用聚類算法（Clustering Algorithm）

聚類分析是無監督學習的一種，嘗試把相似數據收集到一組、把不相似數據收集到另一組[21]。

四種聚類方法[21]：

方法	說明	常見算法
密度法（Density Method）	認為密集區域的點比低密度區域的點有更多相似性與差異；準確度好，且能合併聚類	DBSCAN、OPTICS
層級法（Hierarchical Method）	以樹狀結構形成聚類；新聚類由先前形成的聚類構成	CURE、BIRCH
網格法（Grid-based Method）	把數據表述為有限數量的單元格、形成網格狀結構	CLIQUE、STING
分區法（Partitioning Method）	把對象分區為 k 個聚類，每個分區形成一個聚類	CLARANS

7.4 相關係數（Correlation Coefficient）

相關係數（r）描述散點圖上 x/y 變量之間線性關係的**強度與方向**；r 的值恆介乎 -1 與 +1 之間[21]：

r 值	描述	關係
-1.00	完美下坡（Perfect downhill）	負線性關係
-0.70	強下坡	負線性關係
-0.50	中等下坡	負線性關係
-0.30	弱下坡	負線性關係
0	——	無線性關係
+0.30	弱上坡	正線性關係
+0.50	中等上坡	正線性關係
+0.70	強上坡	正線性關係
+1.00	完美上坡（Perfect uphill）	正線性關係

8. 機器學習的數學基礎

8.1 ML 數學總覽

機器學習涉及的主要數學分支[22]：

- 線性函數（Linear Functions）：線性即「直」；線性函數是一條直線；線性圖代表線性函數

- 線性圖形（Linear Graphics）：圖形在數學、統計學與機器學習中都扮演重要角色
- 線性代數（Linear Algebra）：數據科學的基石（bedrock），掌握線性代數能提升理解數據科學算法的能力
- 概率（Probability）：某事發生或為真的可能性。示例：袋中有 6 球（3 紅、2 綠、1 藍），蒙眼抽一球，抽中綠球的概率＝可發生方式數（2 個綠球）÷結果總數（6 球）＝ 2/6 = 0.333333…。公式：**概率＝方式÷結果**（Probability = Ways / Outcomes）
- 統計學（Statistics）：關於如何收集、分析、解讀與呈現數據，回答「甚麼最常見？甚麼最被預期？甚麼最正常？」

機器學習＝數學：每一項 ML 成功的背後都有數學；所有 ML 模型都以數學的解法與思想構建。ML 的目的是建立理解「思考」的模型。若你志向以下職業，應專注教程所述的數學概念：數據科學家（Data Scientist）、機器學習工程師（Machine Learning Engineer）、機械人科學家（Robot Scientist）、數據分析師（Data Analyst）、自然語言專家（Natural Language Expert）、深度學習科學家（Deep Learning Scientist）[22]。

8.2 線性代數四大對象

機器學習專家離不開線性代數——ML 大量使用純量（Scalars）、向量（Vectors）、矩陣（Matrices）與張量（Tensors）[23]。

對象	定義	維度
純量（Scalar）	單一數字	0 維
向量（Vector）	數字的數組（1 維數組）	1 維
矩陣（Matrix）	二維數組（行×列）	2 維
張量（Tensor）	N 維矩陣	N 維

向量與矩陣是數據的語言——ML 的大部分工作都以向量與矩陣完成[23]。

在 JavaScript 中[24]：純量寫作常數或變量（const scalar = 1）；數組（const array = [1, 2, 3]）；矩陣是二維數組（const matrix = [[1,2],[3,4],[5,6]]）；向量可寫作只有一列的矩陣（[[1],[2],[3]]）或數組（[1, 2, 3]）；張量是 N 維數組。

8.3 向量（Vectors）

向量是 1 維數組，具有**大小（Magnitude）**與**方向（Direction）**，通常描述運動（Motion）或力（Force）[25]。

- **幾何中的向量**：長度顯示大小、箭頭顯示方向。向量 [3, 2] 表示「向右 3、向上 2」的移動。
- **向量加法**：a + b 的求法是把向量 b 移動到其尾部接上向量 a 的頭部（不改變 b），從 a 的尾到 b 的頭的連線即為 a + b。
- **向量減法**：-a 是 +a 的相反向量——大小相同、方向相反。
- **純量運算**：對向量的所有值加、減或乘以一個純量，例如 [1 1 1] + 1 = [2 2 2]；[1 2 3] + 1 = [2 3 4]；[2 2 2] × 3 = [6 6 6]；[6 6 6] ÷ 3 = [2 2 2]。
- **力（Force）與速度（Velocity）** 都是具有大小與方向的向量[25]。

8.4 矩陣（Matrices）

矩陣是一組數字、一個矩形數組，按**行（Rows）**與**列（Columns）**排列[24]。

- **維度（Dimensions）**：1 行 3 列的矩陣維度為 (1 × 3)；2 行 3 列為 (2 × 3)。
- **方矩陣（Square Matrix）**：行數與列數相同的矩陣；n×n 矩陣稱為 n 階方矩陣。
- **對角矩陣（Diagonal Matrix）**：對角線上有值、其餘為零。
- **純量矩陣（Scalar Matrix）**：對角線元素相等、其餘為零。
- **單位矩陣（Identity Matrix）**：對角線為 1、其餘為 0，符號為 I，相當於矩陣世界的「1」；任何矩陣乘以單位矩陣，結果等於原矩陣。
- **零矩陣（Zero Matrix / Null Matrix）**：全部為零。
- **相等矩陣**：每個元素一一對應相等。
- **負矩陣**：所有元素變號[24]。

矩陣運算（教程以 math.js 函式庫示範）[24]：

- **加法／減法**：兩個矩陣維度相同才能相加／相減，逐元素運算。
- **純量乘法**：把矩陣中每個數字乘以該純量（除法同理）。
- **轉置（Transpose）**：以列換行、以行換列，即沿對角線旋轉矩陣。A = [1 2; 3 4] 的轉置 A^T = [1 3; 2 4]。
- **矩陣乘法**：只有當矩陣 A 的**列數**等於矩陣 B 的**行數**時才能相乘；然後計算「點積」（dot product）——把 A 每列的數字與 B 每行的數字相乘再相加。

矩陣乘法示例[24]：

```
A B C
[1 2 3] × [1 4 7; 2 5 8; 3 6 9] = [14, 32, 50]

(1,2,3) · (1,2,3) = 1×1 + 2×2 + 3×3 = 14
(1,2,3) · (4,5,6) = 1×4 + 2×5 + 3×6 = 32
(1,2,3) · (7,8,9) = 1×7 + 2×8 + 3×9 = 50
```

玫瑰銷售應用例：紅玫瑰每枝 $3、白玫瑰 $4、黃玫瑰 $2；星期一賣出（120, 80, 60）、星期二（90, 70, 40）、星期三（60, 40, 20）。以 [3, 4, 2] 乘以銷售矩陣得 [800, 630, 380]，即三天銷售額分別為 $800、$630、$380，總計 $1,810[24]。

矩陣分解（Matrix Factorization）：AI 需要懂得把矩陣分解；矩陣分解是線性代數的關鍵工具，尤其用於線性最小二乘法[24]。

8.5 張量（Tensors）

張量是 N 維矩陣，是向量與矩陣向更高維度的推廣[26]：

- 純量是 0 維張量（0 個軸、秩 Rank 0）
- 向量是 1 維張量（1 個軸、秩 1）
- 矩陣是 2 維張量（2 個軸、秩 2）

張量在 N 維空間中可擁有的方向數目稱為張量的**秩（Rank）**，記作 R。技術上以上全部都是張量，但一般說「張量」時，多指維度大於 2（R > 2）的矩陣[26]。

【補充說明】在 JavaScript 中，張量運算常用 tensorflow.js 函式庫；在 Python 生態中，張量亦是 PyTorch／TensorFlow 的核心數據結構——此為教程中「張量是 N 維數組」概念的延伸理解。

9. 機器學習的統計學與概率

9.1 統計學的兩大分支

統計學是回答數據問題的工具：甚麼常見（Common）？甚麼被預期（Expected）？甚麼正常（Normal）？概率是多少（Probability）[27]?

- **推斷統計（Inferential Statistics）**：從小型**樣本**量化總體屬性的方法——取樣本數據、對整個總體作出預測。例子：在商店訪問 100 人的樣本是否喜歡朱古力，用推斷統計可預測「91% 的購物者喜歡朱古力」。你以推斷統計從小型數據樣本預測整個領域[27]。
- **描述統計（Descriptive Statistics）**：對一組數據的觀察進行總結（描述）。例子：由於每個新生嬰兒都有登記，可知 100 個嬰兒中 51 個是男孩，從而預測新生嬰兒是男孩的概率為 51%。（教程附註：這個比例自 17 世紀以來一直傾斜，並非基礎生物學預測的 50%，至今仍是謎。）**原始觀察只是數據，不是真正的知識**；你用描述統計把原始觀察轉化為可理解的數據[27]。

  描述統計的度量分為兩類[27]：

38. **集中趨勢（Tendency / Measures of the Center）**：平均數（Mean）、中位數（Median）、眾數（Mode）

39. **離散程度（Spread / Measures of Variability）**：最小值與最大值（Min and Max）、標準差（Standard Deviation）、方差（Variance）、偏度（Skewness）、峰度（Kurtosis）

9.2 集中趨勢：平均數、中位數、眾數

教程以 11 個數值為例：7, 8, 8, 9, 9, 9, 10, 11, 14, 14, 15[28]。

- **平均數（Mean）**：所有值的平均——把所有值相加再除以值的個數。本例：(7+8+8+9+9+9+10+11+14+14+15) ÷ 11 = 10.3636363636。平均數＝總和÷個數（The Mean is the Sum divided by the Count）。
- **中位數（Median）**：把數值排序後位於中間的值；若中間有兩個數，取其和除以二。
- **眾數（Mode）**：出現次數最多的值。
- **離群值（Outliers）**：「在其他值以外」的值；離群值可能大幅改變平均數。有時我們不使用它們（它們可能是錯誤），或改用中位數或眾數[28]。

9.3 離散程度：方差與標準差

- **方差（Variance）**：與平均值之差的平方的平均數；換言之，方差描述一組數字偏離平均值的程度[29]。
- **標準差（Standard Deviation）**：衡量數字分散程度的指標，符號為 σ（希臘字母 sigma），公式為方差的平方根（√variance）。偏差（Deviation）是距離的度量——所有值平均而言離平均值（中間）多遠[29]。

9.4 分佈（Distribution）

常態分佈（Normal Distribution） 是一條鐘形曲線（Bell Curve），又稱高斯分佈（Gaussian Distribution）。曲線上每一段的寬度為 1 個標準差（自平均值起計）[30]：

距平均值的距離	佔總體百分比
1 個標準差以內	68.27%
2 個標準差以內	95.45%
3 個標準差以內	99.73%

意義：大部分觀察值在平均值 1 個標準差之內；幾乎所有觀察值在 2 個標準差之內；實際上所有觀察值在 3 個標準差之內。這就是 **68–95–99.7 法則**（又名經驗法則，The Empirical Rule）[30]。

常態分佈的事實[30]：

- 常態分佈是**對稱**的——峰值總把分佈分為兩半。
- 常態分佈是一種**概率分佈**。
- 許多觀察都服從常態分佈：你的智商（IQ）、體重、身高、薪酬、血壓。
- 常態分佈顯示：接近平均值的值比遠離平均值的值更常見。

誤差範圍（Margin of Error）：統計學家總想以 100% 準確度預測一切，但總會存在不確定性；誤差範圍就是量化這種統計不確定性的數字。不同誤差範圍界定我們相信正確答案所在的不同區間；可接受的誤差範圍是判斷問題，取決於答案有多重要。收集的樣本越多，誤差範圍越低[30]。

如何解讀誤差範圍：假設樣本中 55% 的人表示打算投「贊成」票。推及整個總體時，把誤差範圍加減到比例上，得出可能結果的區間——誤差範圍 3% 時，你有信心 52% 至 58% 會投贊成；誤差範圍 10% 時，區間為 45% 至 65%[30]。

偏度（Skewness）：相對鐘形曲線（常態分佈）的扭曲（不對稱）[30]。

峰度（Kurtosis）：同樣是相對常態分佈的扭曲；偏度描述**一條尾部**的異常值，峰度描述**兩條尾部**的異常值。負峰度＝低於常態分佈；正峰度＝高於常態分佈[30]。

9.5 概率（Probability）

概率是關於某事**發生的可能性**或**為真的可能性**；數學概率是介乎 0 與 1 之間的數字——0 表示不可能（Impossibility）、1 表示確定（Certainty）[31]。

事件概率公式：概率＝事件可發生的方式數 ÷ 可能結果的總數（Probability = # of Ways / Outcomes）[31]。

教程示例[31]：

- **擲硬幣**：兩個可能結果——正面（Heads）P = 1/2 = 0.5；反面（Tails）P = 1/2 = 0.5。
- **擲兩枚硬幣**：4 個可能結果，各事件 P(A) = 1/4 = 0.25（正正、反反、正反、反正）。
- **擲骰子**：6 個可能結果，每點 P = 1/6 = 0.166666。同時擲出 3 個「4」的概率是 $(1/6)^3$；同時擲出 3 個相同點數的概率大 6 倍：$(1/6)^3 \times 6$。
- **6 球問題**：袋中 6 球（3 紅、2 綠、1 藍），蒙眼抽中綠球的概率＝2/6 = 0.333333，寫作 P(green) = 0.333333。P(red) = 3/6 = 0.5；P(blue) = 1/6 = 0.1666666。
- **概率比較**：P(A) = P(B) 表示兩事件機會相同；P(A) > P(B) 表示 A 機會較高；P(A) < P(B) 表示 A 機會較低。例如 P(red) > P(green)（抽中紅球比綠球可能性高）。
- **抽中皇牌（King）**：一副牌中抽中 K 的概率＝4/52 = 0.076923，寫作 P(king) = 0.076923。

10. AI 與機器學習的編程語言

機器學習與人工智能涉及的編程語言[32]：LISP、R、Python、C++、Java、JavaScript、SQL。

語言	要點
LISP	世界第二古老的編程語言（1958年），比 Fortran（1957）年輕一年。「Artificial Intelligence」一詞由發明 LISP 的 John McCarthy 創造。LISP 建基於遞歸函數（Recursive Functions，自我修改的函數）理論，非常適合以「自學習」為重要組成部分的機器學習程式
R	用於圖形與統計計算的編程語言，由 R Foundation for Statistical Computing 支持；提供大量統計與圖形技術：線性建模、非線性建模、統計檢驗、時間序列分析、分類、聚類
Python	通用編碼語言，可用於所有類型的編程與軟件開發，典型用於伺服器開發（如構建 Web 應用程式）與數據科學。優勢是擁有非常合適的函式庫：NumPy（數組）、SciPy（統計科學）、Matplotlib（繪圖）、NLTK（自然語言工具包）、TensorFlow（機器學習）
C++	有「世界最快編程語言」之稱；因速度優勢，是編寫電腦遊戲的首選語言——執行更快、響應時間更短，應用於搜索引擎與遊戲開發。Google 在 SEO（搜索引擎優化）的 AI 與 ML 程式中使用 C++。SHARK 是超快的 C++ 函式庫，支持監督學習算法、線性回歸、神經網絡與聚類；MLPACK 也是超快的 C++ 機器學習函式庫
Java	另一種通用編碼語言；在機器學習中主要用於建立算法與神經網絡
SQL	結構化查詢語言（Structured Query Language），管理數據最流行的語言；懂得 SQL 數據庫、表與查詢有助數據科學家處理數據；非常適合在數據庫中儲存、操作與檢索數據

【補充說明】來源頁面另引用 Octoverse 的數據說明 Python 的流行度；上表「Python」一欄的函式庫清單照錄原教程。

11. JavaScript 中的機器學習

11.1 為甚麼 JavaScript 適合 ML

傳統上機器學習應用使用 R 或 Python，但 JavaScript 作為 ML 語言前途亮麗[33]：

- JavaScript 廣為人知，所有開發者都能使用
- 安全性內建——JavaScript 無法訪問你的檔案
- JavaScript 比 Python 快
- JavaScript 可使用硬件加速（hardware acceleration）
- JavaScript 在瀏覽器中運行

機器學習可能數學繁重，神經網絡的技術性與術語往往令人卻步；JavaScript 以易於理解的軟件簡化建立與訓練神經網絡的過程。有了新的 ML 函式庫，JavaScript 開發者可以把機器學習與人工智能加入 Web 應用程式[33]。

瀏覽器中的機器學習意味著：JavaScript 中的 ML、Web 的 ML、人人可用的 ML、更多平台上的 ML。優勢[33]：

- 易用——無需安裝任何東西
- 強大圖形——瀏覽器支持 WebGL
- 更佳私隱——數據可留在客戶端
- 更多平台——JavaScript 可在流動裝置運行

11.2 JavaScript ML 函式庫

函式庫／工具	說明
Brain.js	JavaScript 函式庫，隱藏數學複雜性，令神經網絡易於理解；使用簡單，無需詳知神經網絡細節；提供多種神經網絡實現，不同的網絡可訓練來做好不同的事[33]
ml5.js	致力於讓更廣泛受眾接觸機器學習，以友善方式包裝 ML 功能。示例只用三行程式碼分類圖像：以 ml5.imageClassifier('MobileNet') 建立分類器，再 classifier.classify(...) 對 img 元素分類，於 gotResult(error, results) 回調取得結果[33]

函式庫／工具	說明
TensorFlow Playground	以 d3.js 編寫的 Web 應用程式，讓你在不懂數學的情況下學習神經網絡——在自己的瀏覽器中建立神經網絡並觀察結果[33]
TensorFlow.js	前稱 Tf.js 與 Deeplearn.js（詳見第 12 節）[33]
Math.js	面向 JavaScript 與 Node.js 的廣泛數學函式庫，強大易用，內置大量函數、靈活的表達式解析器，支持數字、大數、複數、分數、單位、數組與矩陣等數據類型[33]
繪圖函式庫	HTML Canvas、Plotly.js、Chart.js、Google Chart、D3.js，可用於 ML 圖形與其他 HTML 圖表[33]
WebGL API	在任何瀏覽器渲染 2D 與 3D 圖形的 JavaScript API，可在集成與獨立顯卡上運行，把 3D 圖形帶入瀏覽器；主要瀏覽器廠商 Apple（Safari）、Google（Chrome）、Microsoft（Edge）、Mozilla（Firefox）均為 WebGL 工作組成員[33]

11.3 Brain.js 實例

Brain.js 建立與使用神經網絡的方式[34]：

- 以 `new brain.NeuralNetwork()` 建立神經網絡
- 以 `network.train([examples])` 訓練網絡；例子代表 4 個輸入值與對應的輸出值
- 以 `network.run([1,0])` 提問：「[1,0] 的可能輸出是甚麼？」——網絡的答案例如：one：93%（接近 1）、zero：6%（接近 0）

預測顏色對比示例：CSS 顏色以 RGB 設定（黑 RGB(0,0,0)、黃 RGB(255,255,0)、紅 RGB(255,0,0)、白 RGB(255,255,255)、淺灰 RGB(192,192,192)、深灰 RGB(65,65,65)）。以 `network.run([0,0,128/255])` 問「深藍色的可能輸出是甚麼？」，網絡答案：Dark（暗）95%、Light（亮）4%——即網絡學會了預測顏色的明暗[34]。

12. TensorFlow.js

12.1 甚麼是 TensorFlow.js？

TensorFlow 是流行的機器學習 JavaScript 函式庫，讓我們在**瀏覽器**中訓練與部署機器學習、為任何 Web 應用程式加入 ML 功能。TensorFlow 由 Google Brain 團隊為 Google 內部使用而開發，2015 年作為開放軟件發布；2019 年 1 月，Google 開發者發布 TensorFlow.js——TensorFlow 的 JavaScript 實現，設計上提供與原 Python 版 TensorFlow 相同的功能[35]。

12.2 張量與基本操作

TensorFlow.js 的主要數據類型是**張量（Tensor）**，與多維數組大同小異，包含一個或多個維度中的值。張量的三大屬性[35]：

屬性	描述
dtype	數據類型
rank	維度數目
shape	每個維度的大小

機器學習中「dimension（維度）」一詞有時與「rank（秩）」互換使用：[10, 5] 是 2 維（2-rank）張量；而「dimensionality」也可指單一維度的大小——[10, 5] 中第一維的 dimensionality 是 10[35]。

張量以 `tf.tensor()` 方法從任何 N 維數組建立，亦可附加 shape 參數（如 `tf.tensor([1,2,3,4], [2,2])`）；以 `tensor.data()` 或 `tensor.array()` 取回背後數據[35]。

張量運算[36]：

- 加法：`tensorA.add(tensorB)`
- 減法：`tensorA.sub(tensorB)`
- 乘法：`tensorA.mul(tensorB)`
- 除法：`tensorA.div(tensorB)`
- 平方：`tensor.square()`
- 重塑：`tensor.reshape()`——張量中元素數目是 shape 各尺寸的乘積；不同 shape 可有相同大小，故常把張量重塑為大小相同的其他 shape

12.3 模型、層與典型工作流

模型（Models）與層（Layers）是機器學習的重要積木；針對不同的 ML 任務，須把不同類型的層組合成可用數據訓練、以預測未來值的模型。**TensorFlow 模型是具有一個或多個層的神經網絡**[37]。

一個 TensorFlow 項目的典型工作流[37]：

40. 收集數據（Collecting Data）
41. 建立模型（Creating a Model）
42. 為模型添加層（Adding Layers）
43. 編譯模型（Compiling）
44. 訓練模型（Training）
45. 使用模型（Using the Model）

教程示例：假設存在直線函數 Y = 1.2X + 5，訓練一個 TensorFlow.js 模型根據 X 輸入預測 Y 值——**模型並不知道這個函數**，而是從數據中學習[37]。各步驟要點[37]：

- **收集數據**：建立含 5 個 x 值的張量（xs），以及 5 個正確 y 答案的張量（ys）——x 乘 1.2 再加 5。
- **建立模型**：建立順序模型（sequential model）——在順序模型中，一層的輸出就是下一層的輸入。
- **添加層**：向模型添加一個密集層（dense layer）；該層只有一個單位（unit，張量）、形狀為 1（一維）。在密集層中，每個節點都連接到前一層的每個節點。
- **編譯模型**：以 meanSquaredError（均方誤差）為損失函數（loss function）、sgd（隨機梯度下降，stochastic gradient descent）為優化器（optimizer）。
- **訓練模型**：以 xs 與 ys 訓練，重複 500 次（epochs，回合）。
- **使用模型**：訓練後可作多種用途，例如給定 10 個 x 值預測 10 個 y 值，並以函數把預測繪成圖形或顯示出來。

TensorFlow 支持的優化器[37]：Adadelta、Adagrad、Adam、Adamax、Ftrl、Nadam、Optimizer（Keras 優化器基類）、RMSprop、SGD（隨機梯度下降優化器）。

12.4 完整示例的數據處理

教程的兩個完整示例（示例 1：汽車——馬力對每加侖英里數 MPG；示例 2：房屋——房間數對價格）都執行 4 個重要步驟：收集數據、準備數據、訓練模型、評估模型[38,39]。

數據收集：示例 1 的數據是汽車對象清單，數據集為 JSON 檔案，存放於 https://storage.googleapis.com/tfjs-tutorials/carsData.json[40]。

清洗數據：準備機器學習時，務必移除不需要的數據、清除數據中的錯誤[40]：

- **移除數據**：聰明做法是隻提取需要的數據——用 map 函數迭代數據，例如從對象的 Horsepower 與 Miles_per_Gallon 屬性只返回 x 與 y。
- **移除錯誤**：大部分數據集都含某些錯誤；聰明做法是用 filter 函數過濾——若 x 或 y 任一屬性為 null 即返回 false。
- **提取數據**：`const jsonData = await fetch("cardata.json"); let values = await jsonData.json(); values = values.map(extractData).filter(removeErrors);`

打亂數據（Shuffle）：訓練前必須打亂數據。訓練時數據被分成小批次（batches），逐批餵給模型；打亂可防止模型重複取得相同數據——若同一數據用兩次，模型將無法泛化（generalize）數據、給出正確輸出；打亂令每批數據更具多樣性[41]。

數據正規化（Normalization）：數據在用於神經網絡前應正規化；對數值數據，使用最小-最大（min-max）縮放到 0–1 範圍通常最佳[41]。

序列模型與層：`const model = tf.sequential();` 建立序列 ML 模型——輸入直接流向輸出（其他模型可有多輸入多輸出）；序列模型最容易，讓你逐層建立模型、權重對應下一層。`model.add()` 添加層；`tf.layers.dense` 是適用於大多數情況的層類型——把輸入乘以權重矩陣、再加上偏置。`inputShape: [1]` 因為只有 1 個輸入（如 x = 馬力或房間數）；`units: 1` 定義權重矩陣大小——每個輸入 1 個權重。編譯：`model.compile({loss: 'meanSquaredError', optimizer: 'sgd'})`——sgd 優化器簡單易用且頗有效；meanSquaredError 用以比較模型預測與真實值[41]。

訓練與測試：`epochs` 定義模型迭代的次數；`model.fit` 是執行循環的函數；`callbacks` 定義模型要重繪圖形時調用的回調函數。模型訓練後必須測試與評估——方法是檢查模型對一系列不同輸入的預測；在此之前須把數據**反正規化（un-normalize）**：以 tf.linspace(0, 1, 100) 產生輸入、model.predict 取得預測，再按 min-max 的逆運算（乘以 max-min 再加 min）還原，然後把 [values, predicted] 繪圖比較[42]。

12.5 TensorFlow.js Visor（tfjs-vis）

TensorFlow Visor 是可視化機器學習的圖形工具[43]：

- 包含可視化 TensorFlow 模型的函數
- 可視化內容可組織在 Visor（模態瀏覽器視窗）中
- 可與 d3、Chart.js、Plotly.js 等自訂工具並用
- 常稱為 tfjs-vis——一個小型的瀏覽器內可視化函式庫，專為配合 TensorFlow.js 而設[38]

tfjs-vis 支持散點圖（Scatter Plots）、長條圖（Bar Graphs）與線性圖（Line Graphs），每種都可在 Visor 模態視窗中組織呈現[43]。

13. JavaScript 圖形函式庫

教程介紹可用於 AI 圖形與其他圖表的 JavaScript 函式庫[44,33]：

函式庫	特點
HTML Canvas	適合散點圖、線性圖、以及散點與線的組合。教程示範建立 Plotter 物件：加入畫線方法、XY 值轉換方法、畫點方法；把程式碼存入函式庫檔案（如 myplotlib.js）即可在 HTML 頁面使用。擁有 Plotter 物件令 AI 更有趣、更可視化、更易理解[45]
Plotly.js	圖表函式庫，提供超過 40 種圖表類型、3D 圖表、統計圖形與 SVG 地圖；支持散點圖、線性圖、多線圖、長條圖（含橫向）、餅圖、甜甜圈圖（加 hole 參數）、方程繪圖[46]
Chart.js	免費的 JavaScript 函式庫，用於製作基於 HTML 的圖表，是最簡單的 JS 可視化函式庫之一。內置圖表類型：散點圖、折線圖、長條圖、餅圖、甜甜圈圖、氣泡圖、面積圖、雷達圖、混合圖。用法：先加入 CDN 連結，再加入具唯一 id 的 `<canvas>` 元素[47]
Google Chart	從簡單折線圖到複雜層級樹狀圖（tree maps），提供大量即用圖表類型：散點圖、折線圖、長條／直條圖、面積圖、餅圖、甜甜圈圖、組織圖（Org Chart）、地圖／地理圖。用法：加入 charts loader 連結、放置具唯一 id 的 `<div>`、載入 Visualization API 與 corechart 套件、設定 API 載入後的回調函數。把 google.visualization.BarChart 換成 PieChart 即可把長條圖轉為餅圖；加 is3D: true 可顯示 3D 餅圖[48]
D3.js	根據數據操作 HTML 的 JavaScript 函式庫；示例以 script 選取 body 元素並附加含「Hello World!」文字的段落；亦示範散點圖[49]

14. AI 應用實例

14.1 圖像分類示例

教程提供互動式「點擊分類圖片」（Click to Classify Picture）與「點擊更換圖片」（Click to Change Picture）的圖像分類示範[50]。

14.2 人工智能音樂（Artificial Music Intelligence）

算法能否比人類譜出更好的音樂? David Cope 是加州聖克魯斯大學（University of Santa Cruz）前音樂教授，30 多年來一直開發 Emmy（又名 EMI，Experimental Musical Intelligence，實驗音樂智能）——以著名作曲家風格作曲的算法[50]。

俄勒岡大學（University of Oregon）Douglas Hofstadter 教授進行的一項測試：鋼琴家演奏三首巴赫風格的樂曲——一首由巴赫（Bach）所作、一首由 Steve Larson 所作、一首由 EMI 所作。當聽眾判定 Larson 的作品是 EMI 所寫時，Larson 博士感到受傷；而當聽眾認為 EMI 所寫的樂曲是真正的巴赫作品時，他感到好些（來源：New York Times）[50]。

14.3 Project Baseline

Project Baseline 是一項倡議，讓每個人更容易為「人類健康地圖」作出貢獻、參與臨床研究。研究人員、臨床醫生、工程師、設計師、倡導者與志願者可在其中協作，構建下一代醫療工具與服務[50]。

14.4 數據科學家

數據科學家可以是多個學科的專家：應用數學、計算統計、電腦科學、機器學習深度學習；並具備大量大數據經驗：商業智能（Business Intelligence）、數據庫設計、數據倉庫設計、數據挖掘、SQL 查詢、SQL 報表[50]。

14.5 人工智能醫療（Artificial Health Intelligence）

冠狀病毒大流行推高了優化醫療保健的需求。機器學習是新技術，可提供更好的藥物發現、更短的開發時間與更低的藥物成本；ML 讓醫療保健利用「大數據」作出更好的醫療或臨床決策[50]。

美國 FDA 局長 Scott Gottlieb 醫生的聲明（關於為 AI 醫療設備建立新的針對性審評框架）：「人工智能與機器學習有潛力從根本上改變醫療保健的交付方式。隨著技術與科學進步，我們可以預期更早的疾病檢測、更準確的診斷、更有針對性的治療，以及個性化醫療的顯著改進。」[50]

15. AI 與社會

15.1 工作取代（Job Replacement）

2013 年，牛津大學科學家 Carl Benedikt Frey 與 Michael A. Osborne 發表研究，指在二十年內 **47% 的所有職業**可能被電腦取代的概率（來源： Oxford - The_Future_of_Employment.pdf）。教程按「電腦化概率」列出各職業的排名[51]：

職業類別	高概率職業（電腦化概率）
簿記	報稅員 99%、會計文員 99%、簿記文員 98%、信用分析員 98%
銷售與客服	電話銷售員 99%、收銀員 97%、推銷員 94%、保險銷售代理 92%、銷售代表 85%、財務顧問 58%、客戶服務 55%、銷售代理 54%
生產工人	包裝機操作員 98%、印刷裝訂工 95%、工業卡車操作員 93%、生產工人 92%、食物製備工人 87%
讀寫工作	數據輸入員 99%、校對員 84%、圖書館員 65%、影視編輯 54%
餐飲	快餐廚師 81%、調酒師 77%、洗碗工 77%
郵政	郵政職員 95%、郵件分揀員 79%、郵政局長 75%、郵差 68%
飛行員	機師 55%
運輸	的士司機 69%、貨車司機 69%、巴士司機 67%

教程的評論要點[51]：

- Frey 與 Osborne「說對了」——今天簿記已自動化，對這些工作而言，電腦比人便宜得多。
- 網購正在接管零售；電話銷售的轉化率不吸引。
- 裝配線機械人已存在 50 年；今天智能機械人可由任何人編程做任何事；機械人正在取代 Amazon 的工作崗位。
- 校對軟件無處不在——從拼寫檢查到 Hemingway App。
- 咖啡機械人可取代許多咖啡師；皇家加勒比「海洋量子號」（Quantum of the Seas）上已有機械人調酒師。

- 軍方今天已使用無人機；將來 AI 會取代貨機機師；客機將只有一名機師，副機師由自動駕駛擔任。
- 自動駕駛汽車已是現實；可能失去的工作崗位數量驚人——AI 很快可能取代數以百萬計的運輸工作。
- 根據牛津經濟研究院（Oxford Economics，2019）的研究，到 2030 年中國可能有 1,400 萬部機械人投入運作；機械人到 2030 年可能取代 2,000 萬個工作崗位。

15.2 心智理論（Theory of Mind）

心智（Mind）是甚麼？ 並無公認定義。心智可定義為智能的實例化（instantiation of intelligence）、知識的集合。思考、感覺與意義是心智還是技術知識（know-how）？心智只是一部大電腦嗎[52]?

認知科學（Cognitive Science） 是研究心智過程的學科；認知科學家研究智能與行為，聚焦腦細胞如何處理與轉換信息，並嘗試學習如何開發智能的電腦算法[52]。

心智模型（The Mental Model）：外部現實的內部圖畫。科學家預期模型在推理與決策（認知）中扮演主要角色。Kenneth Craik 於 1943 年提出：心智在嘗試預料事件時會構建現實的「小尺度模型」（small-scale models）。Jay Wright Forrester（1971）：「我們腦海中 carried 的周圍世界圖像，只是一個模型。沒有人能在腦中想像整個世界、政府或國家。他只有精選的概念、以及概念之間的關係，並用它們代表真實系統。」[52]

AI 能否成為人類？ 科學家正嘗試找出人類智能與人工智能的分別。教程給出的發展時間線[52]：

年份	階段
2000 年	反應式機器（Reactive Machines）
2015 年	機器學習（Machine Learning）
2030 年	心智理論（Theory of Mind）
2050 年	自我意識（Self-Awareness）

- **反應式機器**：早期 AI 系統是反應式的，不能使用過去的經驗。1997 年反應式機器 IBM Deep Blue 擊敗國際象棋世界冠軍——它不能思考，但儲存了棋盤信息與棋子移動規則，勝出因為被編程計算每一步以求勝[52]。
- **機器學習**：今天的 AI 系統能使用部分過去的信息。例子是自動駕駛汽車——它們把預先編程的信息與學習駕駛期間收集的信息結合[52]。
- **心智理論**：心理學術語，指個體的同理心（empathy）與理解他人的能力——意識到他人像自己一樣，有個別的需要與意圖。語言使用者能夠就非具體事物（需要、

意念、概念）溝通。英國心理學家、劍橋大學教授 Simon Baron-Cohen（1999）主張「心智理論」必先於語言出現，依據是早期人類活動：教導、建立共享目標、建立共享計劃、意向性溝通、意向性分享話題、意向性分享焦點、意向性說服、意向性假裝、意向性欺騙[52]。

- **自我意識**：在心理學中，「心智理論」意味人有影響其行為的思想、感覺與情緒。未來的 AI 系統必須學會理解每個人（人類與 AI 對象）都有思想與感覺，必須懂得調整自己的行為，才能行走於我們中間。AI 成為人類前的最後一步是**機器意識（machine consciousness）**——在我們對人類大腦、記憶與智能有更多瞭解之前，無法構建這種軟件[52]。

16. 重點學習總結

46. **AI 的定義與層級**：人工智能是機器模仿人類說話、思考、學習、規劃、理解的能力；狹義 AI（弱 AI）局限於特定任務（如垃圾郵件過濾、語音助手），強 AI 是能複製人類智能、具自我意識的理論層次；AI ⊃ 機器學習 ⊃ 深度學習。
47. **ML 的核心範式**：傳統編程是「數據＋算法＝結果」，機器學習是「數據＋結果＝算法」；模型生命週期為數據收集→訓練→推斷；學習的四種方式為監督、無監督、強化、自監督。
48. **關鍵術語**：標籤（要預測的 y）、特徵（輸入 x）、權重（w）、偏置（b）、關係式 y = b + wx；成本函數（均方誤差）衡量擬合優劣，梯度下降沿成本函數更新權重與偏置，學習率控制每次調整幅度。
49. **感知機是神經網絡的積木**：二進制輸入×權重→求和→與閾值比較→激活輸出；透過反向傳播調整權重從錯誤中學習；單個感知機只能學習線性可分模式，多層感知機加非線性激活可學習複雜模式。
50. **數據佔 ML 項目約 80%**：定量與定性、普查與抽樣、隨機樣本與抽樣偏差、大數據與數據挖掘；訓練前須清洗（map 提取、filter 去錯）、打亂（shuffle）與正規化（min-max 至 0–1）。
51. **數學基礎**：線性函數 y = ax + b（斜率、截距）；線性代數四大對象——純量（0 維）、向量（1 維，具大小與方向）、矩陣（2 維，行×列；加法、純量乘法、轉置、點積乘法、分解）；張量（N 維，秩＝維度數）。
52. **統計與概率**：描述統計（平均數、中位數、眾數；方差、標準差）與推斷統計（由樣本推總體）；常態分佈的 68–95–99.7 法則；誤差範圍、偏度與峰度；概率＝方式÷結果，介乎 0（不可能）與 1（確定）。
53. **歷史脈絡**：從巴比倫六十進制、算盤、Babbage 機械電腦、Zuse Z3（1941）、真空管→晶體管→集成電路，到 1950 年 Turing 論文、1956 年 McCarthy 創「AI」一詞、1997 年 Deep Blue、2015 年 AlphaGo；AI 如今興起的原因是算力、儲存、大數據與快速互聯網同時就緒。

54. **工具生態**：ML 語言包括 LISP、R、Python、C++、Java、JavaScript、SQL；JavaScript 讓 ML 走進瀏覽器——Brain.js（易用的神經網絡）、ml5.js（友善包裝）、TensorFlow.js（Google Brain 出品，張量/層/模型/優化器/epochs 訓練，tfjs-vis 可視化）、math.js 與 Canvas/Plotly/Chart.js/Google Chart/D3.js 繪圖。
55. **AI 與社會**：Frey 與 Osborne（2013）估計二十年內 47% 職業面臨電腦化；牛津經濟研究院（2019）估計機械人到 2030 年或取代 2,000 萬個工作崗位；心智理論路線圖——反應式機器（2000）→機器學習（2015）→心智理論（2030）→自我意識（2050）；機械人技術與 AI 是兩回事，機械人不是 AI 的子集。

來源列表

[1] W3Schools, “Artificial Intelligence (What is AI)”, https://www.w3schools.com/ai/ai_whatis.asp

[2] W3Schools, “AI Tutorial / Machine Learning Intro”, https://www.w3schools.com/ai/

[3] W3Schools, “Human Intelligence (History of Intelligence)”, https://www.w3schools.com/ai/ai_history_brain.asp

[4] W3Schools, “The History of Languages”, https://www.w3schools.com/ai/ai_history_languages.asp

[5] W3Schools, “The History of Numbers”, https://www.w3schools.com/ai/ai_history_numbers.asp

[6] W3Schools, “The History of Computing”, https://www.w3schools.com/ai/ai_history_computers.asp

[7] W3Schools, “History of Robots”, https://www.w3schools.com/ai/ai_robots.asp

[8] W3Schools, “The History of AI”, https://www.w3schools.com/ai/ai_history.asp

[9] W3Schools, “Deep Learning (Neural Networks)”, https://www.w3schools.com/ai/ai_neural_networks.asp

[10] W3Schools, “ML Terminology”, https://www.w3schools.com/ai/ai_ml_terminology.asp

[11] W3Schools, “Machine Learning (Learning / Gradient Descent)”, https://www.w3schools.com/ai/ai_learning.asp

[12] W3Schools, “Machine Learning Data”, https://www.w3schools.com/ai/ai_data.asp

[13] W3Schools, “Scatter Plots”, https://www.w3schools.com/ai/ai_scatter_plots.asp

[14] W3Schools, “Linear Graphs”, https://www.w3schools.com/ai/ai_linear_graphs.asp

[15] W3Schools, “Perceptrons”, https://www.w3schools.com/ai/ai_perceptrons.asp

[16] W3Schools, “Pattern Recognition”, https://www.w3schools.com/ai/ai_recognition.asp

[17] W3Schools, “Training a Perceptron”, https://www.w3schools.com/ai/ai_training.asp

[18] W3Schools, “Testing a Perceptron”, https://www.w3schools.com/ai/ai_perceptron_testing.asp

[19] W3Schools, “Linear Regressions”, https://www.w3schools.com/ai/ai_regressions.asp

[20] W3Schools, “Linear Functions”, https://www.w3schools.com/ai/ai_linear.asp

[21] W3Schools, “Data Clusters (Clustering)”, https://www.w3schools.com/ai/ai_clustering.asp

[22] W3Schools, “ML Mathematics”, https://www.w3schools.com/ai/ai_mathematics.asp

[23] W3Schools, “Linear Algebra”, https://www.w3schools.com/ai/ai_algebra.asp

[24] W3Schools, “Matrices”, https://www.w3schools.com/ai/ai_matrices.asp

[25] W3Schools, “Vectors”, https://www.w3schools.com/ai/ai_vectors.asp

[26] W3Schools, “Tensors”, https://www.w3schools.com/ai/ai_tensors.asp

[27] W3Schools, “Machine Learning Statistics”, https://www.w3schools.com/ai/ai_statistics.asp

[28] W3Schools, “Descriptive Statistics”, https://www.w3schools.com/ai/ai_statistics_descriptive.asp

[29] W3Schools, “Statistic Variability (Variance)”, https://www.w3schools.com/ai/ai_variance.asp

[30] W3Schools, “Distribution”, https://www.w3schools.com/ai/ai_distribution.asp

[31] W3Schools, “Probability”, https://www.w3schools.com/ai/ai_probability.asp

[32] W3Schools, “Machine Learning Languages”, https://www.w3schools.com/ai/ai_languages.asp

[33] W3Schools, “Machine Learning in JavaScript”, https://www.w3schools.com/ai/ai_javascript.asp

[34] W3Schools, “Brain.js”, https://www.w3schools.com/ai/ai_brainjs.asp

[35] W3Schools, “TensorFlow.js Tutorial”, https://www.w3schools.com/ai/ai_tensorflow_intro.asp

[36] W3Schools, “TensorFlow Operations”, https://www.w3schools.com/ai/ai_tensorflow_operations.asp

[37] W3Schools, “TensorFlow Models”, https://www.w3schools.com/ai/ai_tensorflow_model.asp

[38] W3Schools, “TensorFlow Example 1”, https://www.w3schools.com/ai/ai_tensorflow_example_2d.asp

[39] W3Schools, “TensorFlow Example 2”, https://www.w3schools.com/ai/ai_tensorflow_example_houses.asp

[40] W3Schools, “Example 1 Data”, https://www.w3schools.com/ai/ai_tensorflow_data.asp

[41] W3Schools, “Example 1 Model”, https://www.w3schools.com/ai/ai_tensorflow_modelling.asp

[42] W3Schools, “Example 1 Training”, https://www.w3schools.com/ai/ai_tensorflow_training.asp

[43] W3Schools, “TensorFlow.js Visor”, https://www.w3schools.com/ai/ai_tensorflow_visor.asp

[44] W3Schools, “JavaScript Graphics”, https://www.w3schools.com/ai/ai_graphics.asp

[45] W3Schools, “HTML Canvas”, https://www.w3schools.com/ai/ai_canvas.asp

[46] W3Schools, “Plotly.js”, https://www.w3schools.com/ai/ai_plotly.asp

[47] W3Schools, “Chart.js”, https://www.w3schools.com/ai/ai_chartjs.asp

[48] W3Schools, “Google Chart”, https://www.w3schools.com/ai/ai_google_chart.asp

[49] W3Schools, “D3.js”, https://www.w3schools.com/ai/ai_d3js.asp

[50] W3Schools, “Machine Learning Examples”, https://www.w3schools.com/ai/ai_examples.asp

[51] W3Schools, “Job Replacement”, https://www.w3schools.com/ai/ai_job_replacement.asp

[52] W3Schools, “Theory of Mind”, https://www.w3schools.com/ai/ai_mind.asp